Software Architecture and Design Specification

Project: Voice Assistant for MultiMedia Controls

Version: 1.0

Authors: C K Gagan Gowda (PES2UG24CS124), Anudeepak Anpuraja (PES2UG24CS905), Nishkaa V (PES2UG24CS907), Ishanika Vinuthan (PES2UG24CS911)

Date: 3-10-2026

Status: Draft

Revision History

Version | Date | Author | Change Summary

Approvals

Role | Name | Signature/Date

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Voice Assistant for MultiMedia Controls.

1.2 Scope

Covers voice-based control of multimedia playback (play, pause, resume), volume (up, down, mute/unmute), and track navigation (next, previous), along with the supporting voice processing, feedback, logging, and hardware communication.

1.3 Audience

Developers, QA engineers, security auditors, instructors, and maintenance teams.

1.4 Definitions

MQTT, UART, HUD, TLS, STRIDE, UML, RTM.

2. Document Overview

2.1 How to use this document

Provides the architecture and design of the Voice Assistant for MultiMedia Controls, including the component diagram, sequence diagrams, API design, security architecture (STRIDE), and traceability to requirements.

2.2 Related Documents

Software Requirements Specification (SRS) for the Voice Assistant for MultiMedia Controls, including its Requirements Traceability Matrix (RTM).

3. Architecture

3.1 Goals & Constraints

Goals

The architecture of the Voice Assistant for MultiMedia Controls is designed with the following goals:

- Provide hands-free multimedia control using voice commands.
- Support media operations such as Play, Pause, Resume, Volume Up, Volume Down, Mute/Unmute, Next Track, and Previous Track.
- Maintain end-to-end response latency of ≤ 1.5 seconds under normal conditions.
- Achieve at least 90% voice-command recognition accuracy in typical indoor noise conditions up to 60 dB.
- Process audio and video locally to protect user privacy.
- Provide clear audio, visual, or textual feedback after command execution.
- Maintain a modular architecture so that voice processing, command processing, media control, feedback, logging, and hardware communication can be developed and tested independently.

Constraints

- The system requires continuous access to the microphone and, where required, the webcam.
- Voice recognition may be affected by ambient noise, while gesture recognition depends on proper lighting and camera visibility.
- Continuous video processing may increase CPU usage, battery consumption,

and device temperature.

- Gesture tracking may become less reliable beyond approximately 1.5–2 metres from the camera.
- The software must operate on Windows 10/11, macOS, and Ubuntu Linux with Python 3.8+.
- Raw audio and video must not be transmitted to external cloud services.
- A wake word or trigger must be detected before multimedia commands are executed.
- Wi-Fi communication between the host and ESP32/Raspberry Pi must use secure communication such as WPA2/TLS.

3.2 Stakeholders & Concerns

End User

Concerns: Easy hands-free control, fast command execution, accurate recognition, clear feedback, accessibility, and privacy.Developer / System Maintainer

Concerns: Modular architecture, maintainability, easy debugging, clear separation between voice-processing and media-control modules, and useful local logs.

QA Engineer / System Integrator

Concerns: Testability of components, reliable hardware/software integration, command accuracy, latency, error handling, security, and cross-platform compatibility.

Instructor / Assessment Evaluator

Concerns: Compliance with the SRS, traceability of requirements to design components, correct implementation of functional and non-functional requirements, and successful system testing.

3.3 Component (UML) Diagram



3.4 Component Descriptions

1. Microphone Interface

Captures voice commands from the connected microphone and forwards the audio input to the voice-processing module.

2. Voice Processing Engine

Processes incoming audio and performs speech recognition using a local speech-recognition engine. It converts spoken input into recognizable commands.

3. Command Validator / Parser

Validates the recognized command, checks the required wake word or trigger, and maps valid commands to the appropriate multimedia operation. Invalid commands are rejected safely.

4. Media Controller

Handles multimedia operations such as Play, Pause, Resume, Volume Up, Volume Down, Mute/Unmute, Next Track, and Previous Track.

5. Media Command Interface

Connects the system to the operating system or media player and converts internal commands into actual media-control events.

6. Feedback Handler

Provides audio, visual, or textual feedback after a command is processed and informs the user when a command is not recognized.

7. HUD / MediaPipe Interface

Displays the live webcam feed, detected hand landmarks, recognized commands, and current media or volume state. MediaPipe is used for real-time hand-landmark detection.

8. Hardware / Communication Interface

Manages communication between the host system and the ESP32/Raspberry Pi using USB/UART or Wi-Fi-based communication such as MQTT/WebSockets.

9. Logger / Error Handler

Handles invalid or unrecognized commands and records command metadata, timestamps, and execution status for debugging. Raw audio and video are not stored.

3.5 Chosen Architecture Pattern and Rationale

A **layered, pipeline-based modular architecture** was chosen for the Voice Assistant for MultiMedia Controls. Input capture (Microphone Interface, HUD/MediaPipe Interface), processing (Voice Processing Engine, Command Validator/Parser), control (Media Controller, Media Command Interface), and support functions (Feedback Handler, Logger/Error Handler, Hardware/Communication Interface) are each isolated into independent components connected through well-defined interfaces, as shown in the component diagram (3.3).

This pattern was selected because:

- It allows voice processing, gesture/hardware input, and media control logic to be developed, tested, and debugged independently (a key concern for the Developer/Maintainer stakeholder).
- It supports the modularity goal explicitly stated in Section 3.1.
- It keeps the system lightweight and suitable for a single-host, offline-first deployment, unlike a distributed or microservices pattern.

A microservices or event-driven distributed architecture was rejected as unnecessary overhead for a single-user, single-device system with no need for independent scaling, service discovery, or network-distributed deployment. A simple monolithic (non-modular) script-based design was also rejected, as it would violate the separation-of-concerns goal and make independent testing of voice recognition versus media control difficult.

3.6 Technology Stack & Data Stores

Layer	**Technology**
Host runtime	Python 3.8+
Audio capture	PyAudio
Speech recognition	Vosk / SpeechRecognition / Whisper (local, offline models)
Gesture/visual processing	MediaPipe ML SDK (hand-landmark detection)
Media control execution	OS-native media hooks via pyautogui / pynput

Microcontroller communication	pySerial (USB/UART at 115200 baud) and/or MQTT/WebSockets over Wi-Fi
Microcontroller firmware	ESP32 / Raspberry Pi (C++/MicroPython or Python, per hardware choice)
Concurrency	Thread-safe Python queues/events (IPC between capture, processing, and execution threads)
Logging	Local flat-file / CSV debug logs (timestamped command metadata only)

Data stores: The system does not use a persistent database. No cardholder-style sensitive data is involved. The only stored data is a local debug log (command type, timestamp, execution status), which is overwritten/rotated locally and never transmitted externally — consistent with NF-3, NF-6, PRJ-SR-001, and PRJ-SR-004.

3.7 Risks & Mitigations

Risk	**Mitigation**
Ambient noise causes false or missed voice command recognition	Wake-word gating (PRJ-SR-002) and configurable sensitivity thresholds before command parsing
Poor lighting or camera obstruction degrades gesture/hand-landmark tracking	HUD provides live visual feedback so the user can reposition; gesture control treated as a secondary, non-blocking input path
Multiple people or background objects in frame cause erratic gesture tracking	Command Validator/Parser rejects ambiguous or low-confidence input rather than acting on it (fail-safe behavior per Security Objective 4)
Continuous webcam/audio capture increases CPU load, heat, and battery drain	Processing scoped to necessary frame rate (720p@30fps minimum); video pipeline can be disabled when gesture control is not required

Peripheral access conflicts (e.g., mic/webcam locked by another app)	Logger/Error Handler detects and reports peripheral access failures to the user via Feedback Handler
Wi-Fi link between host and ESP32/Raspberry Pi is intercepted or spoofed	Mandatory WPA2/TLS encryption on the Hardware/Communication Interface (PRJ-SR-003)
Unrecognized/unsupported commands trigger unintended media actions	Command Validator/Parser enforces fail-safe default: no action on unrecognized input (FR-11, Security Objective 4)
Gesture tracking unreliable beyond 1.5–2m from camera	Documented operating range communicated to the user; not treated as a critical-path failure

3.8 Traceability to Requirements

Requirement	**Design Component**
FR-01 (Accept voice input)	Microphone Interface
FR-02, FR-03 (Recognize play/pause/resume commands)	Voice Processing Engine → Command Validator/Parser
FR-04–FR-09 (Pause, volume, mute, next/previous)	Media Controller → Media Command Interface
FR-10, FR-11 (Feedback, unrecognized command notice)	Feedback Handler
FR-12 (Predefined command set)	Command Validator/Parser
FR-13 (Execute commands on media player)	Media Command Interface → Connected Media Player
FR-14 (Control without direct player interaction)	Media Controller
FR-15 (Operate on selected hardware platform)	Hardware/Communication Interface → ESP32/Raspberry Pi

NF-1 (Latency ≤ 1.5s)	Voice Processing Engine, Command Validator/Parser (pipeline timing)
NF-2 (≥90% recognition accuracy)	Voice Processing Engine
NF-3 (Local-only processing)	Voice Processing Engine, HUD/MediaPipe Interface
NF-4 (Accessible feedback)	Feedback Handler
NF-5 (Cross-platform)	Host runtime / all components (OS-independent design)
NF-6 (Debug logging)	Logger/Error Handler
PRJ-SR-001 (Local processing only)	Voice Processing Engine, HUD/MediaPipe Interface
PRJ-SR-002 (Wake word/trigger gating)	Command Validator/Parser
PRJ-SR-003 (Encrypted comms)	Hardware/Communication Interface
PRJ-SR-004 (Metadata-only logs)	Logger/Error Handler
PRJ-SR-005 (Local-only HUD access)	HUD/MediaPipe Interface

3.9 Security Architecture

Threat Modeling (STRIDE):

- **Spoofing** — An unauthorized sound or gesture triggering a command is mitigated by requiring a distinct wake word/trigger gesture (PRJ-SR-002) before any command reaches the Media Controller.
- **Tampering** — Command injection via the ESP32/Raspberry Pi link is mitigated by encrypting the Hardware/Communication Interface channel (WPA2/TLS, PRJ-SR-003) and validating all inputs through the Command Validator/Parser before execution.
- **Repudiation** — The Logger/Error Handler records timestamped command metadata and execution status, providing an audit trail without storing raw

audio/video (NF-6, PRJ-SR-004).

- **Information Disclosure** — All audio/video processing is performed locally with no transmission to external/cloud services (NF-3, PRJ-SR-001), preventing leakage of captured voice or video data.
- **Denial of Service** — Resource conflicts (e.g., webcam/mic locked by another process) or excessive false-trigger processing are handled gracefully by the Logger/Error Handler, which reports failures via the Feedback Handler rather than allowing the pipeline to hang or crash.
- **Elevation of Privilege** — The HUD/debug interface, which exposes the live camera feed, is restricted to local-device access only and is never exposed over the network (PRJ-SR-005), preventing remote access to sensitive visual data or system state.

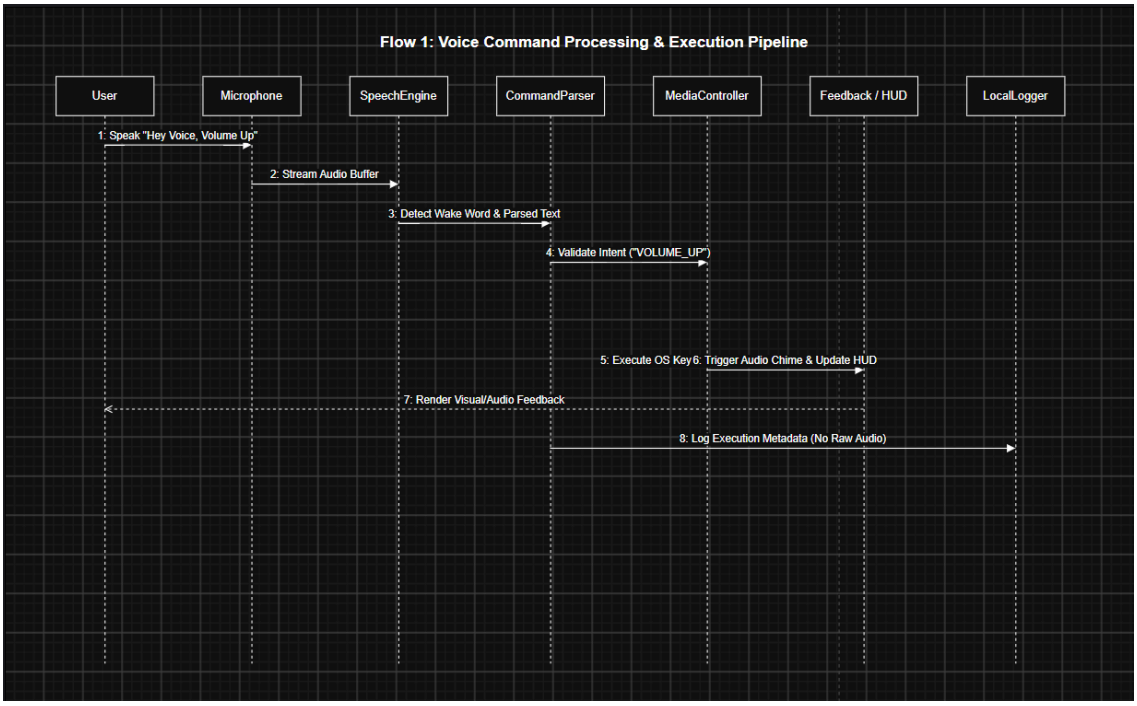
4. Design

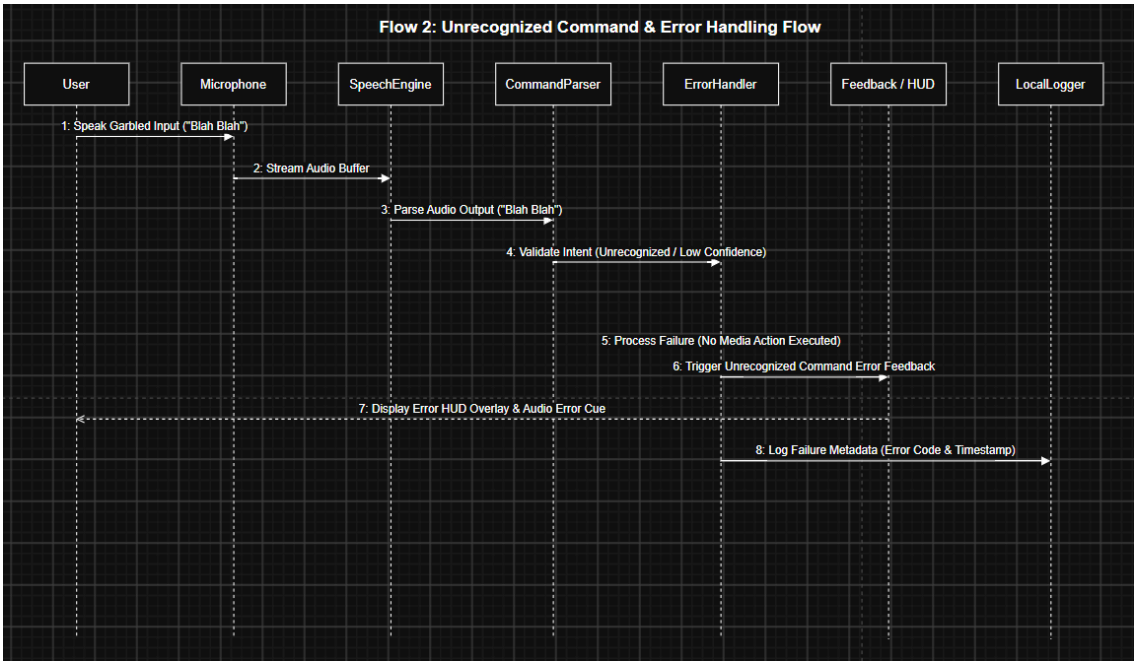
The system is designed with a layered architecture to ensure clear separation of concerns, modularity, and ease of maintenance. It bridges multimodal user inputs (voice and visual gestures) to system-level OS media events through three primary operational layers:

- **Input & Perception Layer:** Continuously captures raw audio buffers via PyAudio from the microphone array and video frames via OpenCV/MediaPipe from the webcam.
- **Processing & Core Logic Layer:** Comprises local speech processing engines (Vosk/Whisper) and hand landmark tracking pipelines. It handles wake-word detection, intent parsing, command validation, and failure routing.
- **Execution & Output Layer:** Maps recognized intent structures to local system-level OS media event hooks (e.g., pyautogui, pynput), delivers accessible audio/visual feedback via HUD overlay, and logs timestamped execution metadata to local files without persisting raw streams.

4.2 UML Sequence Diagrams

At least 2 sequence diagrams covering 2 flows specific to your project





4.3 API Design

Interface Definition 1: Voice Command Processing

Endpoint: /voice/process

Method: POST
Request: {audioChunk, sampleRate, wakeWordDetected}
Response: {status, recognizedText, intent, confidence}
Errors: 401 Wake word missing, 422 Unrecognized command.

Interface Definition 2: Media Command Execution

Endpoint: /media/execute
Method: POST
Request: {intent, stepIncrement, commandSource}
Response: {status, executedAction, feedbackTriggered}
Errors: 400 Invalid command, 503 OS media hook failed.

4.4 Error Handling, Logging & Monitoring

Error Handling Strategy

- **Fail-Safe Processing:** Unrecognized, ambiguous, or low-confidence voice inputs are intercepted by the ErrorHandler (UC-10, UC-09) and result in no action on the target media player, preventing unintended media disruptions.
- **Standardized Error Structures:** All internal component errors emit structured error objects containing a standardized error code, human-readable message,

Standardized Logging Rules

- **Strict Privacy Enforcement (PRJ-SR-004 & NF-6):** Raw audio buffers, binary voice recordings, and video frame matrices are never written to disk or transmitted across any network interface.

Monitoring & Operational Metrics

The system collects real-time local metrics to ensure operational performance compliance:

- **Command Latency Monitoring (NF-1):** Tracks end-to-end execution latency from audio input to OS action execution to maintain 90th percentile latency under ≤ 1.5 seconds.
- **Transaction & Recognition Failure Rates (NF-2):** Monitors ratio of successful intent resolutions against total wake-word activations to track command recognition accuracy ($\ge 90\%$ threshold).
- **Resource & Peripheral Health:** Tracks webcam and microphone input stream locks to gracefully handle hardware access conflicts with external applications.

4.5 UX Design

The primary interface is a minimalist Head-Up Display (HUD) overlay showing the live webcam feed, hand-landmark tracking, recognized voice commands, and volume/media state. Distinct audio cues confirm each executed command for eyes-free use. Accessibility is supported through high-contrast gesture boundary markers, adjustable HUD scale, customizable voice/gesture sensitivity thresholds, and spoken confirmation toggles.

4.6 Open Issues & Next Steps

Future enhancements: expanded voice command set and secondary IR transmitter control via the ESP32/Raspberry Pi. Open issue: gesture-tracking reliability beyond 1.5–2 metres from the camera.

5. Appendices

5.1 Glossary: MQTT (Message Queuing Telemetry Transport), UART (Universal Asynchronous Receiver Transmitter), HUD (Head-Up Display), TLS (Transport Layer Security), STRIDE, UML, RTM.

5.2 References: SRS for the Voice Assistant for MultiMedia Controls (v1.0); IEEE 42010; MediaPipe, Vosk, and pySerial documentation.

5.3 Tools: Python 3.8+, PyAudio, Vosk / SpeechRecognition / Whisper, MediaPipe, pyautogui / pynput, pySerial (see 3.6).